

11. Write a program to demonstrate function overloading (same function name, different parameters)**Code:**

```
#include <iostream>
using namespace std;

// Function overloading: same function name, different parameters
void display(int a) {
    cout << "Integer: " << a << endl;
}

void display(double d) {
    cout << "Double: " << d << endl;
}

void display(string s) {
    cout << "String: " << s << endl;
}

int main() {
    display(10);      // Calls the int version
    display(3.14);    // Calls the double version
    display("Hello!"); // Calls the string version

    return 0;
}
```

OUTPUT:

```
Integer: 15
Double: 2.77
String: Hello world!
```

12. Write a program to implement function overloading**Code:**

```
#include <iostream>
using namespace std;

// Function overloading: same function name, different parameter types
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

string add(string a, string b) {
    return a + b;
}

int main() {
    // Calling overloaded functions with different types of parameters
    cout << "Sum of integers: " << add(5, 3) << endl;
    cout << "Sum of doubles: " << add(2.5, 3.7) << endl;
    cout << "Concatenation of strings: " << add("Hello, ", "World!") << endl;
    return 0;
}
```

OUTPUT:

```
Sum of integers: 13
Sum of doubles: 10.2
Concatenation of strings: Hello, World!
```

13. Write a program to implement a class and object with data members and member functions.

Code:

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length;
    double width;

public:
    void setDimensions(double l, double w) {
        length = l;
        width = w;
    }

    double getArea() {
        return length * width;
    }

    void display() {
        cout << "Length: " << length << endl;
        cout << "Width: " << width << endl;
        cout << "Area: " << getArea() << endl;
    }
};

int main() {

    Rectangle rect1;

    rect1.setDimensions(6.0, 9.0);

    rect1.display();

    return 0;
}
```

OUTPUT:

A screenshot of a terminal window showing the output of the program. The text is displayed on a black background with a light blue border on the left. The output consists of three lines: "Length: 6", "Width: 9", and "Area: 54".

```
Length: 6
Width: 9
Area: 54
```

14. Write a program to show the concept of constructors and destructors.**Code:**

```
#include <iostream>
using namespace std;

class Car {
private:
    string model;
    int year;

public:
    Car(string m, int y) {
        model = m;
        year = y;
        cout << "Constructor called: Car model " << model << " created!" << endl;
    }

    ~Car() {
        cout << "Destructor called: Car model " << model << " destroyed!" << endl;
    }

    void display() {
        cout << "Car Model: " << model << endl;
        cout << "Year: " << year << endl;
    }
};

int main() {
    Car myCar("Toyota", 2022);

    myCar.display();

    return 0;
}
```

OUTPUT:

```
Constructor called: Car model mercedes created!
Car Model: mercedes
Year: 2020
Destructor called: Car model mercedes destroyed!
```

15. Write a program to implement function overloading**Code:**

```
#include <iostream>
using namespace std;

// Function overloading: same function name, different parameters

// Overloaded function to add two integers
int add(int a, int b) {
    return a + b;
}

// Overloaded function to add two doubles
double add(double a, double b) {
    return a + b;
}

// Overloaded function to add three integers
int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    // Calling overloaded functions
    cout << "Sum of integers: " << add(5, 3) << endl;    // Calls int version
    cout << "Sum of doubles: " << add(2.5, 3.7) << endl;    // Calls double version
    cout << "Sum of three integers: " << add(1, 2, 3) << endl; // Calls int version with 3
    params

    return 0;
}
```

OUTPUT:

```
Sum of integers: 11
Sum of doubles: 6.2
Sum of three integers: 6
```

16. Write a program to implement single inheritance.**Code:**

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
    void eat() {
        cout << "Animal is eating." << endl;
    }
};

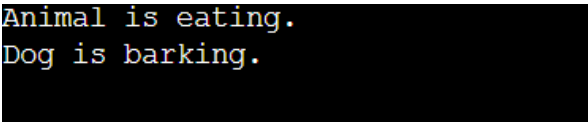
// Derived class (inherits from Animal)
class Dog : public Animal {
public:
    void bark() {
        cout << "Dog is barking." << endl;
    }
};

int main() {
    // Creating an object of the derived class Dog
    Dog dog;

    // Calling function from the base class Animal
    dog.eat(); // Inherited from Animal

    // Calling function from the derived class Dog
    dog.bark(); // Specific to Dog

    return 0;
}
```

OUTPUT:

```
Animal is eating.
Dog is barking.
```

17. Write a program to implement multiple inheritance.**Code:**

```
#include <iostream>
using namespace std;

// Base class 1
class Animal {
public:
    void eat() {
        cout << "Animal is eating." << endl;
    }
};

class Vehicle {
public:
    void drive() {
        cout << "Vehicle is driving." << endl;
    }
};

class Robot : public Animal, public Vehicle {
public:
    void work() {
        cout << "Robot is working." << endl;
    }
};

int main() {
    // Creating an object of the derived class Robot
    Robot robot;

    robot.eat(); // Inherited from Animal
    robot.drive(); // Inherited from Vehicle
    robot.work(); // Specific to Robot

    return 0;
}
```

OUTPUT:

```
Animal is eating.
Vehicle is driving.
Robot is working.
```

19. Write a program to demonstrate hierarchical inheritance.**Code:**

```
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Animal is eating." << endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Dog is barking." << endl;
    }
};

class Cat : public Animal {
public:
    void meow() {
        cout << "Cat is meowing." << endl;
    }
};

int main() {
    Dog dog;
    Cat cat;

    dog.eat(); // Inherited from Animal
    dog.bark(); // Specific to Dog

    cat.eat(); // Inherited from Animal
    cat.meow(); // Specific to Cat

    return 0;
}
```

OUTPUT:

```
Animal is eating.
Dog is barking.
Animal is eating.
Cat is meowing.
```

20. Write a program to show the use of this pointer.**Code:**

```
#include <iostream>
using namespace std;

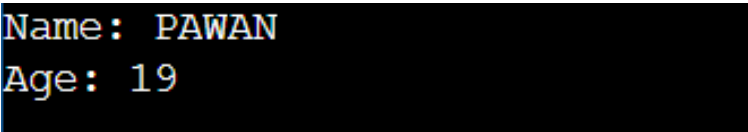
class Student {
private:
    string name;
    int age;

public:
    // Member function using 'this' pointer
    void setData(string name, int age) {
        this->name = name; // 'this' refers to the current object's member
        this->age = age;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main() {
    Student s1;
    s1.setData("Alice", 20);
    s1.display();

    return 0;
}
```

OUTPUT:

```
Name: PAWAN
Age: 19
```

21. Write a program to demonstrate deep copy using a copy constructor (important when using pointers).

Code:

```
#include <iostream>
#include <cstring>
using namespace std;

class Person {
private:
    char* name;

public:

    Person(const char* n) {
        name = new char[strlen(n) + 1];
        strcpy(name, n);
    }

    Person(const Person& other) {
        name = new char[strlen(other.name) + 1];
        strcpy(name, other.name);
    }

    void display() {
        cout << "Name: " << name << endl;
    }

    ~Person() {
        delete[] name;
    }
};

int main() {
    Person p1("John"); // Original object
    Person p2 = p1;    // Copy constructor is called (deep copy)

    p1.display();
    p2.display();

    return 0;
}
```

OUTPUT:

```
Name: pawan
Name: pawan
```